Does Spec-Driven Development Reduce Defects? An Empirical Test of Industry Claims Across 119 Open-Source Repositories

Brenn Hill

Independent Researcher

Working Paper — April 2026

Suggested citation: Hill, B. (2026). Does Spec-Driven Development Reduce Defects? An Empirical Test of Industry Claims Across 119 Open-Source Repositories. SSRN Working Paper.

Abstract

Context. Spec-driven development (SDD) tools claim that writing specifications before implementation reduces defects, prevents rework, and improves code quality. No production-scale SDD tool data is publicly available. **Objective.** We provide the first large-scale empirical test of five hypotheses derived from SDD vendor claims, using specification artifacts in open-source pull requests — the closest available proxy — scored on the same quality dimensions SDD tools prescribe. **Method.** We analyze 88,052 pull requests across 119 open-source repositories (103 with SZZ defect tracing coverage), using within-author fixed-effects estimation, propensity score matching, and ten robustness checks. **Results.** None of the five hypotheses are supported. The raw association between specifications and defects is reversed — spec'd PRs introduce *more* defects, not fewer. Within-author controls point to confounding by indication: developers spec their hardest work, and hard work produces defects. Specification quality predicts neither rework ($p = 0.827$) nor AI scope constraint ($p = 0.997$). One result trends in the predicted direction — higher quality correlates weakly with fewer defects ($p = 0.016$, 8 fewer per 1,000 PRs per 10-point quality gain) — but does not survive robustness checks. **Conclusion.** Specification artifacts proxy for task complexity, not quality improvement.

1 Introduction

The rise of AI-assisted software development has produced a new category of developer tooling: specification-driven development (SDD) frameworks. These tools — including GitHub's Spec Kit (Delimarsky, 2025) and Amazon's Kiro (Brooker, 2025) — claim that writing formal specifications before implementation leads to fewer defects, less rework, and higher-quality code, particularly when AI agents perform the implementation.

Böckeler (Böckeler, 2025) defines spec-driven development as "writing a 'spec' before writing code with AI" where "[t]he spec becomes the source of truth for the human and the AI." The GitHub Copilot Academy frames it more strongly: "Specifications don't serve code — code serves specifications" (GitHub Copilot Academy, 2025). The movement gained momentum in 2025 with the release of GitHub's Spec Kit (open-source) and Amazon's Kiro IDE. Böckeler examined three tools pursuing the approach (Böckeler, 2025). More broadly, SDD is positioned as an antidote to "vibe coding" — iteratively prompting AI without formal requirements — which practitioners associate with scope creep, rework, and defects.

The core premise is intuitive: if developers articulate what software should do before writing it, AI systems will produce more reliable output. The GitHub Copilot Academy lists "reduced rework, living documentation, systematic quality" among SDD's benefits (GitHub Copilot Academy, 2025). Kiro claims that specifications enable "fewer iterations and higher accuracy" and "prevent costly rewrites" (Yanchuleff, 2025). GitHub's Spec Kit blog post promises "less guesswork, fewer surprises, and higher-quality code" (Delimarsky, 2025).

These claims rest on a plausible causal chain: clearer requirements → less ambiguity → fewer defects. Yet vendors cite no empirical evidence for these claims in their marketing materials, documentation, or blog posts. No study has tested SDD's quality claims on production-scale data prior to this paper.

This paper provides the first large-scale empirical test. We analyze 88,052 merged pull requests across 119 open-source repositories, after excluding 12,195 bot-authored PRs. Defects are traced to their introducing commits using the SZZ algorithm. We use within-author fixed-effects estimation to control for developer-level confounding, and derive five hypotheses directly from vendor claims.

2 Related Work

2.1 Requirements Engineering and Defect Prevention

The claim that upfront specification prevents defects has deep roots in software engineering. Boehm's cost-of-change curve (Boehm, 1981) argued that requirements defects are exponentially cheaper to fix early. However, subsequent work has challenged the universality of this finding.

Shull et al. (Shull et al., 2002) found that the cost multiplier depends heavily on project context, and agile methodologies have demonstrated that iterative refinement can be more effective than upfront specification for certain project types (Beck et al., 2001).

The question of whether requirements quality predicts software defects has been studied directly. Mund et al. (Mund et al., 2015) combined two empirical studies and found only weak evidence that requirements specification quality predicts implementation defects, with effect sizes that varied substantially across projects. Femmer et al. (Femmer et al., 2017) developed automated detection of "requirements smells" (ambiguity, incompleteness, inconsistency) and found that while smells were prevalent, their relationship to downstream defects was context-dependent. These findings suggest that the relationship between specification quality and code quality is not as straightforward as SDD vendors assume — a pattern our study confirms at larger scale.

2.2 SDD Literature

The academic literature on SDD perpetuates the pattern of unsubstantiated claims. Piskala (Piskala, 2026) claims "controlled studies showing error reductions of up to 50%," citing a Red Hat Developer blog post (Naszcyniec, 2025) and an InfoQ article (Griffin & Carroll, 2026) as evidence. Neither source contains any controlled study, any experiment, or any quantitative data. The "50%" figure traces through a citation chain that ultimately rests on practitioner opinion pieces with no empirical backing. Guo et al. (Guo et al., 2025) claim SDD "guarantees software quality" without empirical validation. Marri (Marri, 2026) reports a 73% security defect reduction from constitutional specs, but on a single project ($n = 1$), with the same developer implementing both conditions sequentially — the author acknowledges "small sample size limits statistical power." Roy (Roy, 2026) proposes iterative spec-code co-evolution but reports no independently validated quality outcomes. Zietsman (Zietsman, 2026) runs small planted-bug experiments and is honest about their limitations ("directional, not statistically significant"), while contributing the useful observation that AI reviewers without specs suffer circular reasoning — but does not test whether specs reduce defects at scale.

Practitioner evaluations have been more skeptical. Eberhardt, writing for Scott Logic, found Spec Kit roughly ten times slower than iterative development, producing thousands of lines of specification markdown that still resulted in buggy code (Eberhardt, 2025). Zaninotto argued that "SDD shines when starting a new project from scratch, but as the application grows, the specs miss the point more often and slow development," and proposed iterative natural-language development as a faster alternative (Zaninotto, 2025).

3 Hypotheses

We identified five empirical claims made by SDD tool vendors in their public documentation, blog posts, and marketing materials. Each claim is stated verbatim and mapped to a testable hypothesis.

Claim 1: Specifications reduce defects.

GitHub's Spec Kit blog post promises "less guesswork, fewer surprises, and higher-quality code" (Delimarsky, 2025). Kiro claims specifications lead to "fewer iterations and higher accuracy" (Yanchuleff, 2025).

H1: Pull requests with specification artifacts have lower defect-introduction rates than pull requests without them, controlling for author and PR size.

Claim 2: Specifications reduce rework.

The GitHub Copilot Academy lists "reduced rework" among SDD's benefits (GitHub Copilot Academy, 2025). Kiro claims to "prevent costly rewrites" (Yanchuleff, 2025). GitHub's blog states that SDD enables "iterative development without expensive rewrites" (Delimarsky, 2025).

H2: Pull requests with specification artifacts have lower rework rates than pull requests without them, controlling for author and PR size.

Claim 3: Clearer specifications produce more reliable code.

GitHub's Spec Kit blog states that "providing a clear specification up front, along with a technical plan and focused tasks, gives the coding agent more clarity, improving its overall efficacy" (Delimarsky, 2025). Kiro claims specifications enable "fewer iterations and higher accuracy" (Yanchuleff, 2025). This implies a dose-response relationship: higher specification quality should predict fewer defects.

H3: Among pull requests with specifications, higher specification quality predicts lower defect-introduction rates, controlling for author and PR size.

H4: Among pull requests with specifications, higher specification quality predicts lower rework rates, controlling for author and PR size.

Claim 4: Specifications constrain AI-generated code scope.

GitHub's Spec Kit blog describes tasks that give "the coding agent a way to validate its work and stay on track" (Delimarsky, 2025). Kiro claims specs provide "a North Star to guide the work of the agent, allowing it to take on larger tasks without getting lost" (Brooker, 2025).

H5: Specifications reduce code churn (additions + deletions) for AI-tagged pull requests more than for human-authored pull requests, controlling for author.

4 Methods

4.1 Dataset

We collected pull request data from 119 open-source repositories spanning 17 programming languages. Repositories were selected to represent large, active projects with diverse contribution patterns. We collected data between March 28 and April 3, 2026, with a 365-day lookback window per repository. The resulting dataset spans April 2025 through early April 2026 (1st–99th percentile: April 8, 2025 to April 1, 2026), totaling 100,247 PRs. We excluded 12,195 bot-authored

PRs (dependabot, renovate, and other automated accounts identified by username pattern and bot flags), leaving 88,052 PRs from human and AI-assisted authors.

For each pull request, we extracted: author, merge date, additions, deletions, files changed, linked issues, PR description, review comments, and co-author tags (used to identify AI-assisted contributions).

Metric	**Value**	**Notes**
Repositories	119	convenience sample, 17 languages
PRs (post-bot exclusion)	88,052	12,195 bots removed
Median PRs per repo	517	range 9–4,803
Median spec rate (per repo)	26.2%	IQR 16.0–41.1%
Median AI tag rate (per repo)	1.2%	IQR 0.4–8.1%
SZZ-covered repos	103	16 had unreachable merge SHAs
SZZ bug-introducing PRs	9,754	12.5% of SZZ-covered PRs
Quality-scored PRs	5,192	5.9% of total; non-random subsample

Table 1: Study dataset overview.

The sample spans 17 programming languages, with Python (31 repos, 26%), TypeScript (26, 22%), Go (17, 14%), and Rust (13, 11%) predominating. The median repository has a 26.2% specification rate (IQR 16.0–41.1%). AI tagging is present in most repositories: the median AI tag rate is 1.2% (IQR 0.4–8.1%). The 5,989 AI-tagged PRs provide sufficient statistical power for the primary AI analyses, though we recognize that the AI-specific results would benefit from a larger AI-tagged sample as agentic workflows become more prevalent.

4.2 Variables

4.2.1 Specification Presence (`specd`)

A pull request is classified as "specified" (hereafter "spec'd") if it meets any of the following criteria: (a) it links to a GitHub issue or external tracking ticket, (b) its description references a specification document, RFC, or design doc, or (c) its description contains structured requirements content (acceptance criteria, behavioral descriptions, scope boundaries). Classification is performed programmatically by parsing PR metadata: issue references (`#123`, `fixes #456`), cross-reference URLs, and description structure (presence of headings, checklists, or requirement-style language). 24,298 of 88,052 PRs (27.6%) are classified as specified.

This operationalization is deliberately generous: it captures any form of upfront specification artifact, from a linked Jira ticket to a detailed RFC. SDD tools claim benefits from *any* form of specification, so we test the broadest reasonable definition. Specification *presence* and specification *quality* are tested separately: H1 and H2 test whether having any specification artifact

predicts fewer defects or less rework; H3 and H4 test whether, among the subset of PRs that have specifications, higher-quality specifications predict better outcomes. This two-level design addresses the objection that our binary `specd` measure is too coarse by also testing whether the content of the specification matters.

4.2.2 Specification Quality (`q_overall`)

For the subset of specified PRs with substantial descriptions (body length > 50 characters), we scored specification quality by prompting Claude Haiku (Anthropic) with the PR title and body and a structured rubric. The LLM scored each PR across seven dimensions: outcome clarity, error states, scope boundaries, acceptance criteria, data contracts, dependency context, and behavioral specificity. Each dimension is scored 0–100, and the overall quality score is the mean across dimensions.

These seven dimensions were chosen to align with the quality criteria recommended by SDD tools themselves. Spec Kit's template prescribes "core requirements" (outcome clarity), "acceptance criteria" (our acceptance criteria dimension), "edge cases" (error states), "data models" and "API contracts" (data contracts), and "user stories" (behavioral specificity) (Delimarsky, 2025). Kiro prescribes "clear requirements," "acceptance criteria," "system architecture," and "technology choices" (Yanchuleff, 2025). Six of our seven dimensions have direct equivalents in Spec Kit's recommended structure; all seven are covered across both tools. This alignment is deliberate: we measure specification quality on the dimensions that SDD vendors themselves say matter.

We validated the LLM scoring against independent human ratings on a stratified random sample of 38 PRs. The human rater scored each dimension on the same rubric, blind to the LLM scores. Overall rank-order agreement was moderate: Spearman $\rho = 0.42$ ($p = 0.01$), Pearson $r = 0.45$ ($p < 0.01$). The LLM scores systematically higher than the human rater (mean bias: +4 points on the 0–100 scale, mean absolute difference: 13 points). Per-dimension agreement varies: behavioral specificity ($\rho = 0.45$, $p = 0.004$), scope boundaries ($\rho = 0.43$, $p = 0.007$), dependency context ($\rho = 0.40$, $p = 0.01$), and outcome clarity ($\rho = 0.39$, $p = 0.02$) show significant agreement, while error states ($\rho = 0.22$), data contracts ($\rho = 0.19$), and acceptance criteria ($\rho = 0.17$) show weak correlation. The LLM reliably detects whether specification content is *present* but cannot judge whether it is *correct* for the domain — a distinction between formal completeness and functional quality. Since even this completeness-oriented measure shows no defect benefit, the null finding is conservative: a more precise quality measure would sharpen the estimate, not reverse it.

5,192 PRs (5.9% of total) received quality scores. This subsample is non-random — only spec'd PRs with sufficient description text were scored — and results from this subsample carry a selection bias caveat.

A further limitation: we cannot distinguish specifications written before implementation from descriptions written after. Many PR descriptions in our dataset are likely post-hoc — the developer wrote the code, then described what it does. We compared quality scores for PRs with linked issues (more likely pre-implementation intent; median quality 57, $N = 975$) to PRs classified by description content alone (more likely post-hoc; median quality 51, $N = 978$). The difference

is statistically significant ($p < 0.001$) but substantively small (6 points on the 0–100 scale), and the direction varies by dimension. Neither group shows meaningfully different defect rates. The pre/post distinction does not appear to matter for our quality measure, and the null finding holds across both subgroups.

Furthermore, many effective AI instructions are inherently underspecified by our rubric's standards. A prompt like "improve the error handling, it looks ugly" scores low on outcome clarity, acceptance criteria, and behavioral specificity — yet it is a perfectly successful instruction for an AI agent that can read the surrounding code. The agent does not need the spec to enumerate every error case; it can infer them from context. This class of task — where the codebase itself provides sufficient specification — is invisible to our quality measure but may represent a large share of productive AI-assisted work. If detailed specifications add no value for tasks where the code context is sufficient, the marginal benefit of specification effort is even smaller than our results suggest.

4.2.3 Defect Introduction (`szz_buggy`)

We applied the SZZ algorithm (Śliwerski et al., 2005) to 103 of 119 repositories, tracing 64,805 blame links from fix commits to bug-introducing commits. The remaining 16 repositories produced zero traceable blame links. Their merge commit SHAs (recorded by the GitHub API) are not reachable in single-branch clones, typically because squash-merge workflows cause GitHub's synthetic merge commits to be garbage-collected after merging. The SZZ algorithm uses `git blame` to trace defect-fixing commits back to the commits that introduced the defect. We use the basic SZZ variant, which da Costa et al. (Costa et al., 2017) found misattributes 46–71% of bug-introducing changes depending on the project. This noise is substantial, but it is non-differential with respect to specification presence: there is no reason to expect SZZ to systematically misattribute more for spec'd PRs than unspec'd PRs. Non-differential measurement error attenuates associations toward the null, which works against finding a protective effect but does not create spurious reversed associations. We acknowledge the implications for statistical power in Section 7. Each bug-introducing commit was mapped to its originating pull request. A PR is marked `szz_buggy = True` if any of its commits were identified as introducing a defect that was later fixed. 9,754 PRs (12.5% of those in SZZ-covered repos) are marked as bug-introducing.

4.2.4 Rework (`reworked`)

A pull request is classified as reworked if a subsequent PR within 30 days by any author modifies overlapping files and has a title or description indicating correction (matching patterns such as "fix," "revert," "bugfix," "hotfix," "regression," "broke," or "broken"). File overlap is computed by comparing the set of files changed in each PR pair; high-frequency files (touched by >30% of PRs in the repository) are excluded to avoid spurious attribution. 11,820 PRs (13.4%) are classified as reworked. The 30-day window was chosen based on sensitivity analysis: rework detection increases from 14 to 30 days but shows diminishing returns beyond 30 days (the spec–rework association is directionally consistent across 14-, 30-, 60-, and 90-day windows).

4.2.5 AI-Tagged (`ai_tagged`)

A pull request is classified as AI-assisted through two detection layers. First, a regex matches co-author tags (e.g., `Co-authored-by: Copilot`), generation attribution strings (`Generated with`, `Claude Code`, `written with/by` followed by a tool name), AI agent markers (`CURSOR_AGENT`, `coderabbit.ai`, `AI-generated`), and the robot emoji (🤖). Second, we parse structured "AI Disclosure" and "AI Usage" sections that some repositories include in their PR templates, classifying the disclosure content as AI-used or no-AI-used. We validated the combined classifier against 537 PRs with clear self-reported AI disclosures: precision is 98.8% (1 false positive) and recall is 24.2% (75.8% false negatives). The low recall reflects that many developers describe AI use in natural language that our patterns do not capture. 5,989 PRs (6.8%) carry AI tags after bot exclusion. This remains a lower bound.

4.2.6 Code Churn (`total_churn`)

Total code churn is defined as additions + deletions. We use `log(1 + churn)` in regressions to normalize the heavily right-skewed distribution (median: 55, mean: 698).

4.3 Statistical Approach

For each hypothesis, we report three levels of analysis:

Pooled: Raw rates and Fisher's exact test. This is the analysis that unadjusted claims would cite.

Controlled: Logistic regression with size controls (log additions, log deletions, log files changed) and repository fixed effects (dummy variables). This addresses between-repository confounding.

Within-author: Linear probability model (LPM) with full author demeaning and clustered standard errors. All variables — treatment, controls, and outcome — are demeaned by author, equivalent to author fixed effects via the Frisch–Waugh–Lovell theorem. Standard errors are clustered at the author level to account for within-author residual correlation. We use LPM rather than logistic regression because demeaning is exact for OLS but produces biased estimates in logit due to the incidental parameters problem (Neyman & Scott, 1948). Chamberlain's conditional logit avoids this problem but requires discarding groups without outcome variation, reducing power. We follow Angrist and Pischke's recommendation of LPM for fixed-effects estimation with binary outcomes (Angrist & Pischke, 2009). Primary models use size controls (log additions, log deletions, log files changed); we verify that results hold when adding JIT defect prediction features (Kamei et al., 2013) as additional controls (Section 5.6).

The within-author estimate is the most credible. Observational studies of developer practices face severe confounding: developers who write specifications may differ systematically from those who do not in skill, experience, task selection, and organizational context. The within-author design addresses this by comparing the same developer's specified PRs to their own unspecified PRs, eliminating all time-invariant author-level confounding. The treatment effect is identified only from authors who have *both* specified and unspecified PRs in the dataset.

We restrict within-author analysis to authors with ≥2 PRs and report the number of authors with treatment variation (those who have both spec'd and unspec'd PRs), as the treatment effect is identified from these authors' within-author variation. For within-author estimates, we report 95% confidence intervals based on clustered standard errors to allow readers to assess what effect sizes are compatible with the data.

We do not apply multiple-comparison corrections across the five hypotheses. At $\alpha = 0.05$, approximately 0.25 false positives are expected by chance. We interpret all results with this in mind.

Sample sizes vary across hypotheses because of SZZ coverage, quality scoring, and within-author filtering:

Hypothesis	**PRs (pre-filter)**	**Within-author *N***	**Identifying authors**
H1: specs → bugs	77,814 (103 SZZ repos)	72,046	2,181
H2: specs → rework	88,052 (all 119 repos)	81,647	2,524
H3: quality → bugs	5,192 (scored subset)	4,901	357
H4: quality → rework	5,192 (scored subset)	4,901	357
H5: spec × AI → churn	88,052 (all 119 repos)	71,932	366

Table 2: Sample sizes by hypothesis.

Within-author *N* is after restricting to authors with ≥2 PRs and dropping rows with missing controls. Identifying authors are those with variation in both treatment and control conditions.

5 Results

5.1 H1: Specifications Reduce Defects

Analysis restricted to 77,814 PRs in the 103 repositories with SZZ coverage.

Method	**Coefficient**	***p*-value**	**Interpretation**
Pooled (Fisher's exact)	OR = 1.202	< 0.001	Spec'd PRs have *higher* defect rate
Controlled (logit, no repo FE[†])	0.067	0.006	Effect persists with size controls
Within-author (LPM)	+0.014 [+0.005, +0.024]	0.003	+1.4pp *increase* in defect rate

Table 3: H1: Specifications and defect-introduction rates.

[†] Repository fixed effects caused a singular matrix; controlled estimate uses size controls only. Within-author 95% CI in brackets.

Identified from 2,181 authors with treatment variation (out of 4,328 authors with ≥2 PRs).

H1 is not supported. The pooled comparison is the unadjusted analysis: spec'd PRs have a 20.2% higher odds of introducing a defect than unspec'd PRs. But this comparison is confounded — developers who write specs are different from those who don't, and the repos that encourage specs are different from those that don't. The controlled logistic regression adds size controls (repository fixed effects caused a singular matrix and were dropped). The effect persists.

The within-author estimate is the most credible. It compares the *same developer's* spec'd PRs to their own unspec'd PRs, eliminating all time-invariant author-level confounding — skill, experience, coding style, and organizational context are held constant because both the "treatment" and "control" come from the same person.

Within-author, spec'd PRs are associated with a 1.4 percentage-point *increase* in defect-introduction rates ($p = 0.003$, OR = 1.13, Cohen's $d = 0.07$). The direction is opposite to the vendor claim at every level of analysis.

This pattern is called *confounding by indication*: developers rationally invest specification effort in proportion to task complexity, so harder tasks both receive specs and introduce more defects. The specification does not cause the defects — the task difficulty that motivates writing one does. When JIT risk features are added as controls (Section 5.6), the within-author spec coefficient drops 55% and loses significance ($p = 0.229$), confirming that measurable task complexity explains most of the association.

A plausible alternative explanation is detection bias: if spec'd PRs have better test coverage and more granular QA, their defects may be detected and fixed individually, making them more visible to SZZ tracing. Unspec'd PRs might have defects that go undetected longer or get fixed in bulk refactoring commits that SZZ cannot trace, producing artificially low defect rates. We cannot rule this out entirely. However, two observations weigh against it. First, the rework measure (H2) does not depend on SZZ tracing and shows the same directional pattern. Second, propensity score matching (Section 5.6) eliminates the defect association entirely by matching on observable risk features. This suggests task complexity — not detection asymmetry — is the primary driver.

5.2 H2: Specifications Reduce Rework

Analysis includes all 88,052 PRs.

Method	Coefficient	p-value	Interpretation
Pooled (Fisher's exact)	OR = 1.179	< 0.001	Spec'd PRs have higher rework
Controlled (logit + repo FE)	0.178	< 0.001	Effect persists
Within-author (LPM)	+0.012 [+0.005, +0.019]	0.001	+1.2pp *increase* in rework rate

Table 4: H2: Specifications and rework rates.

Identified from 2,524 authors with treatment variation (out of 4,892 with ≥2 PRs).

H2 is not supported. The within-author effect is in the wrong direction: the same author's spec'd PRs have a 1.2 percentage-point higher rework rate than their unspec'd PRs ($p = 0.001$, OR = 1.10, Cohen's $d = 0.05$). The pooled odds ratio (1.18×) is modest, and within-author, the effect is small in magnitude but statistically significant and in the wrong direction. Specifications are associated with more rework, not less.

5.3 H3: Specification Quality Reduces Defects

H1 and H2 tested specification *presence* — whether having any specification artifact matters. H3 and H4 test specification *quality* — whether, among PRs that already have specifications, better-written specifications predict better outcomes. This is the stronger version of the SDD claim: not just "write a spec" but "write a *good* spec." We consider H3 and H4 *exploratory* rather than confirmatory: the quality measure has limited construct validity (human–LLM agreement $\rho = 0.42$ on $N = 38$) and is tested on a non-random 5.9% subsample.

Quality is the mean of seven rubric dimensions (described in Section 4.2), each scored 0–100. These dimensions were chosen to align with the quality criteria that SDD tools themselves prescribe (Section 4.2). The scoring is automated with limited human validation ($\rho = 0.42$ on 38 PRs; Section 4.2) — a limitation we acknowledge (Section 7). The quality score measures *formal* specification completeness (are error states described? are acceptance criteria present?) rather than *functional* correctness (are the right error states described? are the acceptance criteria valid for this domain?). A specification can score highly on every dimension and still specify the wrong behavior.

Analysis restricted to 5,192 PRs with LLM-scored specification quality (6.7% of SZZ-covered PRs). Selection bias caveat: only specified PRs with substantial descriptions were scored.

Method	Coefficient	p-value	Interpretation
Controlled (logit + repo FE)	−0.005	0.045	Quality predicts fewer defects
Within-author (LPM)	−0.0008 [−0.0014, −0.0001]	0.016	−0.08pp per quality point

Table 5: H3: Specification quality and defect-introduction rates.

Identified from 357 authors with treatment variation.

H3 is weakly supported on a biased subsample. The within-author coefficient is statistically significant ($p = 0.016$) but substantively small: a 10-point improvement in specification quality (on the 0–100 scale) predicts a 0.8 percentage-point reduction in defect rate. This is the one result in the SDD-predicted direction. However, three caveats limit its interpretation. First, it is identified from 357 authors on 5.9% of the data, in a subsample non-randomly selected for having substantial specification content. Second, when restricted to the four quality dimensions with validated human–LLM agreement (Section 5.6), the effect weakens ($p = 0.075$) while the unvalidated dimensions strengthen — suggesting the signal may reflect description length or task

complexity rather than specification quality (Section 5.6). Third, the effect size is small enough that even if causal, it would not justify the specification effort required to achieve it.

5.4 H4: Specification Quality Reduces Rework

Method	Coefficient	p-value	Interpretation
Controlled (logit + repo FE)	0.003	0.195	Quality does not predict rework
Within-author (LPM)	−0.000 [−0.001, +0.001]	0.827	Effectively zero

Table 6: H4: Specification quality and rework rates.

Identified from 357 authors with treatment variation (same scored subsample as H3).

H4 is not supported. Specification quality has no meaningful relationship with rework rates within-author. The coefficient is effectively zero, with $p = 0.827$. The controlled logistic regression is also non-significant ($p = 0.195$), consistent with the within-author finding (likely reflecting the same confounding-by-indication pattern: thorough specifications accompany difficult tasks that are more likely to be reworked).

5.5 H5: Specifications Constrain AI Scope

Analysis uses log code churn (additions + deletions) as outcome. PRs with zero churn are excluded (these represent missing data rather than true zero-change events). AI detection combines regex pattern matching with structured AI disclosure parsing (Section 4.2), identifying 5,989 AI-tagged PRs — 2.3× more than co-author tags alone.

Group	Within-author coef	p-value	Interpretation
AI-tagged PRs	+0.120	0.030	Spec'd AI PRs are *larger*
Human PRs	+0.140	< 0.001	Spec'd human PRs are *larger*
Interaction (spec × AI)	+0.000 [−0.101, +0.102]	0.997	No differential scope constraint

Table 7: H5: Specifications and AI-generated code scope.

AI analysis identified from 224 authors with treatment variation; interaction from 366.

H5 is not supported. Rather than constraining scope, specifications accompany *larger* PRs for both AI-tagged (+12.0%) and human-authored (+14.0%) work. The interaction term is effectively zero ($p = 0.997$): specifications have the same relationship with code churn for AI and human PRs alike — the same confounding-by-indication pattern seen in H1 and H2. Harder tasks get specs and produce more code.

5.6 Robustness Checks

Ten additional analyses test whether the null result is an artifact of our primary measures or insufficient controls.

5.6.1 Alternative Outcome Measures

In addition to SZZ-traced defects and rework, we test specification effects on two additional outcome measures: *escaped defects* (PRs whose CI passed but were subsequently followed by a fix or revert) and *strict escaped* (escaped PRs where the follow-up PR's title explicitly indicates a fix or regression).

Outcome	Within-author coef	p-value	N
SZZ bugs	+0.014	0.003	72,046
Rework	+0.012	0.001	81,647
Escaped	+0.001	0.333	81,647
Strict escaped	+0.001	0.108	81,647

Table 8: Alternative outcome measures (within-author LPM).

N reflects within-author estimation restricted to authors with ≥2 PRs; the smaller samples compared to H1 (77,814) and H2 (88,052) reflect this filtering. The SZZ bugs and rework rows reproduce the H1 and H2 findings (included here for comparison). Both show significant positive associations — specs accompany more defects and more rework, not less — which we attribute to confounding by indication. The escaped and strict escaped measures, which capture a different class of defect (CI-passing code that subsequently required a fix), are also directionally positive but not significant. No outcome measure shows specifications reducing defects at $p < 0.05$.

5.6.2 Incremental Validity Beyond JIT Features

The JIT defect prediction framework (Kamei et al., 2013) uses 14 code-change features (subsystems touched, file entropy, developer experience, among others) to predict defect-introducing commits. We test whether specification information adds predictive power beyond JIT features on the 61,192 PRs with both complete JIT features and SZZ outcomes.

Model	Pseudo R^2	AIC	Spec variable p
JIT features only	0.078	44,793	—
JIT + spec presence	0.078	44,786	0.003
JIT + spec quality[†]	0.096	—	0.059

Table 9: Incremental validity of specifications beyond JIT features.

[†] Scored subset only (5,192 PRs). JIT-only Pseudo R^2 on same subset = 0.095.

Spec presence is statistically significant when added to the JIT model ($p = 0.003$) but the increment is substantively negligible: Δ Pseudo $R^2 = 0.0002$, and AIC decreases by only 7

points — a change too small to affect model predictions in practice. Spec quality adds a marginally significant increment ($p = 0.059$) on a biased subsample. In both cases, the incremental predictive power is too small to matter in practice — JIT features account for nearly all the predictive power.

5.6.3 Individual Quality Dimensions

We test each of the seven specification quality dimensions independently against SZZ bugs and rework (within-author LPM, 4,901 PRs):

Dimension	**→ bugs coef**	***p***	**→ rework coef**	***p***
Outcome clarity	−0.001	0.119	−0.000	0.812
Error states	−0.000	0.100	+0.000	0.308
Scope boundaries	−0.001	0.027	−0.000	0.239
Acceptance criteria	−0.001	0.063	−0.000	0.469
Data contracts	−0.001	0.037	+0.000	0.927
Dependency context	−0.000	0.536	+0.000	0.411
Behavioral specificity	−0.001	0.020	−0.000	0.676

Table 10: Individual quality dimensions vs. defects and rework (within-author LPM).

All coefficients in the defects column are negative (directionally protective) but extremely small. To put the magnitudes in plain language: the largest coefficient is −0.001 (scope boundaries). This means that scoring the maximum possible improvement on scope boundaries — moving from 0 to 100 on the rubric — would predict 0.1 fewer percentage points of defects. In a codebase with a 12% base defect rate, that is a reduction from 12.0% to 11.9%. No individual quality dimension produces a practically meaningful reduction in defects. The rework column is even starker: every coefficient rounds to zero.

Three dimensions show marginal statistical significance for defects (scope boundaries $p = 0.027$, data contracts $p = 0.037$, behavioral specificity $p = 0.020$), but none survive Bonferroni correction[1] (threshold: $p < 0.007$ for 7 tests). Statistical significance at these magnitudes reflects the large sample size, not a meaningful effect. Even if these tiny coefficients reflected real causal relationships, they would be too small to justify the specification effort required to achieve them.

5.6.4 Validated Quality Dimensions Only

Our LLM quality rubric was validated against human ratings on 38 PRs (Section 4.2). Four of seven dimensions showed significant human–LLM agreement: behavioral specificity ($\rho = 0.45$), scope boundaries ($\rho = 0.43$), dependency context ($\rho = 0.40$), and outcome clarity ($\rho = 0.39$). Two dimensions showed weak agreement: acceptance criteria ($\rho = 0.17$) and error states ($\rho = 0.22$). If measurement noise in the unvalidated dimensions attenuates the quality coefficient toward zero (classical errors-in-variables bias), restricting to validated dimensions should recover a larger effect.

[1] With 7 simultaneous tests at $\alpha = 0.05$, the Bonferroni-corrected threshold is $p < 0.007$.

Quality composite	→ bugs coef	p	→ rework coef	p
All 7 dimensions	−0.0008	0.016	−0.0001	0.827
4 validated dimensions only	−0.0006	0.075	−0.0001	0.849
3 unvalidated dimensions only	−0.0008	0.006	+0.0001	0.808

Table 11: Validated vs. unvalidated quality dimension composites.

The pattern is not what measurement-error attenuation would predict. The validated-only composite weakens ($p = 0.075$), while the unvalidated composite — the dimensions where LLM and human disagree — strengthens ($p = 0.006$). If the quality score were capturing a real protective effect attenuated by noise, removing the noisy dimensions should sharpen the estimate, not weaken it. The validated and unvalidated composites have similar variance (SD 20.9 vs. 20.8), so the weakening is not explained by a power difference. Still, the strongest signal comes from the dimensions our instrument measures least reliably, which is not the pattern a true quality effect would produce. The unvalidated dimensions likely correlate with description length rather than specification quality: longer descriptions give the LLM more text in which to detect keywords for each dimension, and longer descriptions accompany more complex tasks, which produce more defects. This is confounding by indication operating through the quality measure itself. For rework (H4), all three composites produce coefficients indistinguishable from zero.

5.6.5 Repo-Level Analysis

We also aggregate to the repository level ($N = 98$ repos with ≥50 PRs) and test whether repos with higher specification rates have lower defect or rework rates.

Comparison	Spearman ρ	p-value
Spec rate vs. defect rate	−0.113	0.268
Spec rate vs. rework rate	−0.047	0.643

Table 12: Repo-level specification rate vs. outcome rates.

At the repository level, specification rate has no relationship with either defect rate ($\rho = -0.11$, $p = 0.27$) or rework rate ($\rho = -0.05$, $p = 0.64$). Neither correlation is significant. An OLS regression controlling for mean PR size yields a spec rate coefficient of −0.064 ($p = 0.12$), which is not significant.

5.6.6 Subgroup Analysis

SDD tools are marketed primarily for AI-assisted workflows. We test whether the null result holds across human-only PRs, AI-tagged PRs, and repos stratified by AI adoption rate. As in all analyses, bot PRs are excluded.

Subgroup	→ bugs coef	p	→ rework coef	p
All non-bot PRs (88,052)	+0.014	0.003	+0.012	0.001
Human-only, no AI (82,063)	+0.015	0.003	+0.011	0.004
AI-tagged, non-bot (5,989)	+0.011	0.424	+0.010	0.399
Zero-AI repos (3,562)	−0.016	0.377	−0.033	0.014
Low-AI repos (51,877)	+0.019	0.001	+0.009	0.061
High-AI repos (32,613)	+0.010	0.185	+0.018	0.003

Table 13: Subgroup analysis: specification effects across populations.

The pattern is consistent across subgroups. For human-authored PRs, specs are associated with *more* defects and *more* rework — the confounding-by-indication pattern. For AI-tagged PRs, the coefficients are positive but not significant ($p = 0.42$ for defects, $p = 0.40$ for rework). The null holds whether a repo has zero AI adoption, moderate adoption, or high adoption.

5.6.7 High-Quality Specs Only

A likely reviewer objection is that our `specd` measure is too loose — that linked Jira tickets are not "real" specifications in the SDD sense, and only high-quality structured specs should show the benefit. We test this directly by restricting to top-quartile (quality score ≥ 67) and top-decile (≥ 77) specs. Quality thresholds are computed from the scored subset's distribution (median = 55, p75 = 67, p90 = 77).

Test	→ bugs coef	p	Identifying authors
Top-quartile specs vs. all others	−0.025	0.036	225
Top-decile specs vs. all others	−0.015	0.370	113
High-quality vs. NO spec (low-quality excluded)	−0.027	0.052	184
AI-tagged + high-quality spec	−0.044	0.070	52

Table 14: High-quality specification thresholds vs. defect rates.

The top-quartile result is marginally significant (−2.5pp, $p = 0.036$) but does not survive scrutiny. First, when we perform the cleanest comparison — high-quality specs vs. PRs with *no spec at all*, excluding low-quality specs entirely — the effect is borderline ($p = 0.052$). Second, if there were a real dose-response relationship, top-decile specs should show a *stronger* effect than top-quartile. Instead the effect weakens ($p = 0.370$) — the opposite of what a causal relationship predicts. Third, the top-quartile result would not survive Bonferroni correction across the four tests shown. Fourth, even top-quartile specs do not reduce rework (+0.9pp, $p = 0.578$).

The AI + high-quality spec result ($p = 0.070$, −4.4pp) is the closest test to the agentic SDD workflow vendors are selling. However, it is identified from only 52 authors — far too few for a stable estimate — and the effect vanishes when comparing AI specs broadly ($p = 0.424$). We note it as suggestive but not robust.

The objection that our measure is "too loose" does not rescue the SDD claims. Even restricting to the best-quality specs, the protective effect is fragile, does not follow a dose-response pattern, and does not reduce rework.

5.6.8 JIT Features as Primary Controls

A reviewer concern is that size controls (log additions, deletions, files) are insufficient — the JIT defect prediction framework (Kamei et al., 2013) captures task complexity dimensions (entropy, file age, developer experience) that may confound the specification–defect association. We re-run H1 and H2 with 13 JIT features as additional controls alongside size controls (subsystem experience excluded for zero variance).

Test	Size controls only	1. JIT controls	Change
H1: specs → bugs (coef)	+0.014 ($p = 0.003$)	+0.006 ($p = 0.229$)	−55%
H2: specs → rework (coef)	+0.012 ($p = 0.001$)	+0.009 ($p = 0.024$)	−23%

Table 15: JIT features as primary controls.

For H1, the spec coefficient drops 55% and loses significance when JIT features are added, indicating that much of the apparent spec–defect association is explained by task complexity dimensions that size alone does not capture. For H2, the coefficient drops 23% but remains significant: specifications remain associated with more rework even after controlling for JIT risk features. The within-author spec coefficient in the JIT-controlled model (+0.006, $p = 0.229$) is not significant for defects.

5.6.9 Propensity Score Matching

As an alternative to regression-based controls, we match each spec'd PR to an unspec'd PR with a similar JIT risk profile using nearest-neighbor propensity score matching (caliper = 0.05 SD of the logit propensity score). This directly addresses confounding by indication by comparing spec'd PRs to observationally similar unspec'd PRs.

Outcome	Spec'd (matched)	Unspec'd (matched)	Difference
Defect rate (17,375 pairs)	14.7%	15.2%	−0.6pp ($p = 0.133$)
Rework rate (19,648 pairs)	15.2%	14.7%	+0.5pp ($p = 0.146$)

Table 16: Propensity score matching results.

All 16 covariates are well-balanced after matching (all standardized mean differences < 0.1). For defects, the matched spec'd and unspec'd defect rates are indistinguishable — propensity score matching eliminates the raw association entirely. For rework, the result is the same: after matching on observable risk features, the rework difference is +0.5 percentage points and not significant ($p = 0.146$). Both the defect and rework associations are explained by the JIT risk profile of the tasks that receive specifications.

5.6.10 Temporal Analysis: The SDD Era

Specification adoption in our dataset is not uniform over time. Monthly specification rates rose from 23.7% in April 2025 to 31.3% by March 2026, with AI-tagged PR rates rising from 0.3% to 9.8% over the same period. If SDD tools are driving specification adoption and those specifications improve outcomes, the effect should be visible in the most recent data — the period closest to production SDD usage.

We restrict the within-author analysis to the most recent three months of the observation window (January–March 2026), when specification rates are highest (29.3%) and AI adoption is most prevalent (8.8%):

Test	**Recent 3 months coef**	***p***	**Full dataset coef**	***p***
Specs → SZZ bugs	+0.004	0.435	+0.014	0.003
Specs → rework	+0.016	0.001	+0.012	0.001
AI + specs → bugs	+0.003	0.822	—	—
AI + specs → rework	+0.016	0.212	—	—
Spec × AI interaction (H5)	+0.011	0.861	+0.000	0.997

Table 17: Temporal analysis: recent three months vs. full dataset.

The pattern is consistent. In the period of highest SDD adoption, specifications are associated with more rework (+1.6pp, $p = 0.001$) and directionally more defects (+0.4pp, $p = 0.435$, not significant). For AI-tagged PRs, specs have no effect in either direction. The H5 scope-constraint interaction is null in both the full dataset and the recent window. The null result is not a historical artifact — it persists in the data most representative of the SDD era.

A caveat on the temporal data: SZZ-traced defect rates for the most recent months are likely underestimates, because defects introduced recently may not yet have been fixed (and therefore cannot be traced back to their introducing commits). This right-censoring affects absolute defect rates but not the *relative* comparison between spec'd and unspec'd PRs within the same time window, which is the quantity of interest.

The null result holds across every angle: four outcome measures, two predictive frameworks (LPM and JIT), propensity score matching on JIT risk profiles, seven quality dimensions, two units of analysis (PR-level and repo-level), six subgroup cuts, multiple quality thresholds, and temporal restriction to the period of highest SDD adoption. When JIT features are added as controls, the H1 spec coefficient drops 55% and loses significance; propensity score matching eliminates the association entirely. No specification measure robustly predicts fewer defects or less rework under any operationalization.

6 Discussion

6.1 Confounding by Indication

The pattern across all five hypotheses is identical: unadjusted analysis suggests specifications accompany worse outcomes; within-author analysis attenuates but does not eliminate the reversed association; propensity score matching on JIT features eliminates it entirely, confirming confounding by indication (Salas et al., 1999). Developers rationally invest specification effort in proportion to task complexity. Harder tasks receive specifications *and* produce more defects — the specification does not cause the defects.

JIT risk feature profiles confirm this directly. Spec'd PRs have significantly higher change entropy (1.3×), more lines added (1.6×), touch older files (1.8×), and involve more prior developers (1.2×) than unspec'd PRs (all $p < 0.001$). Notably, spec'd PR authors have *less* experience than unspec'd PR authors (median 62 vs. 121 prior commits, $p < 0.001$) — less experienced developers spec more, not less, consistent with specifications being a support mechanism for harder tasks.

This confounding cuts both ways. The within-author null is consistent with both "specifications have zero effect" and "specifications have a small positive effect masked by residual task selection." We cannot distinguish these interpretations with observational data. However, the within-author estimates are not merely insignificant but *directionally reversed* across most tests, and propensity score matching eliminates both the defect and rework associations entirely. If a protective effect exists, it is smaller than our study can detect.

Spec'd PRs take substantially longer to merge (median 29.4 hours vs. 11.6 hours for unspec'd), but this additional review time does not translate into fewer defects — the same confounding pattern. AI-tagged PRs receive less review scrutiny despite a higher base defect rate (16.7% vs. 12.2%), a separately concerning finding.

6.2 Why Specifications May Not Prevent Defects

One explanation is how many things must go right simultaneously for a specification to prevent defects. The spec must be accurate in every assertion. It must have no substantial omissions. The AI must interpret it exactly as intended. The AI must make no implementation mistakes. And the spec itself must describe the correct solution. Each condition is individually difficult; together they represent a high bar. Defects arise precisely where human understanding is incomplete — and specifications are written by the same humans whose incomplete understanding produces the defects. This does not prove specifications *cannot* work, but it suggests why the marginal benefit may be smaller than vendors assume.

SDD vendors conflate *directing* the AI (telling it what to build) with *ensuring quality* (fewer defects, less rework). Specifications may be effective at direction — a structured task description is better than a vague prompt. But an agent faithfully executing a spec will reproduce every gap,

ambiguity, and wrong assumption in that spec. The specification tells the AI *what* to build. It does not tell the AI *what it forgot to specify*. Direction is not quality.

6.3 Vendor Evidence

No SDD tool vendor has published empirical evidence for the quality claims tested here. The evidence base consists of first-principles reasoning and developer testimonials. GitHub describes Spec Kit as "an experiment" (Delimarsky, 2025) and publishes no effectiveness data. Third-party evaluations are negative: Böckeler found Spec Kit's overhead "overkill for the size of the problem" (Böckeler, 2025) Eberhardt found it roughly ten times slower than iterative development (Eberhardt, 2025). Notably, none of the 119 repositories in our dataset show evidence of SDD tooling — a text search for `.specify/`, `.speckit/`, `.kiro/`, and `spec-kit` across 100,247 PRs returns zero matches. GitHub, Amazon, and other SDD vendors have the data and engineering resources to conduct controlled evaluations of their products. That vendors have not done so — while making specific quality claims in marketing materials — is itself informative.

We measure the *kind* of specification content that SDD tools claim to improve upon, scored on the same quality dimensions those tools prescribe. If the tool's format itself is the active ingredient — something beyond the seven quality dimensions we measure — that would be a testable claim that vendors have not tested.

SDD tooling imposes real costs: specification-writing time, process overhead, organizational change. Teams adopting SDD on the basis of vendor claims are making investment decisions on unsubstantiated assertions. We do not argue that thinking about requirements is wasted effort. We argue that the *products* claiming to reduce defects through specification artifacts have not demonstrated that they do.

7 Threats to Validity

7.1 Construct Validity

Three gaps separate what we measure from what SDD vendors claim.

We detect specification artifacts, not specification processes. Our classifier flags linked issues, structured descriptions, and referenced design documents. A linked Jira ticket created to satisfy a process requirement counts the same as a detailed design document written after iterative refinement. If the benefit comes from the *thinking* rather than the *artifact*, our measure is too coarse. Two observations partially address this concern. First, the within-author design compares the same developer's spec'd and unspec'd work, controlling for that person's consistent habits. Second, H3 and H4 test quality *within* spec'd PRs on seven SDD-aligned dimensions. If deeper thinking produced better artifacts, those artifacts should predict better outcomes. They do not. The remaining objection — that process depth helps through a mechanism invisible in the artifact itself — is difficult to test with any observational design.

We test organic specifications, not SDD tool output. Spec Kit generates structured documents through a guided workflow and feeds them to a coding agent. A linked GitHub issue is not the same thing. Our operationalization is the broadest reasonable proxy available, but it may miss whatever specific mechanism the tools provide. The high-quality spec subsample (top-quartile, scored on the dimensions SDD tools prescribe) is the closest approximation. It too shows no robust effect.

We cannot observe agentic workflows directly. Even for AI-tagged PRs with specifications, we do not know whether the spec was actually used as input to an agent or just written for human consumption. True agentic SDD — where the agent reads the spec, generates code from it, and validates against it — is not observable in our data. The 52-author AI + high-quality spec subsample is the closest proxy we have.

7.2 Internal Validity

Defect detection is noisy. The SZZ algorithm traces fix commits back to their introducing commits via `git blame`. da Costa et al. (Costa et al., 2017) found that basic SZZ misattributes 46–71% of bug-introducing changes. We use the basic variant for implementation simplicity across 103 repositories. This noise is substantial. We initially assumed it to be non-differential with respect to specification status, but this assumption deserves scrutiny: spec'd PRs are 1.6× larger and touch files with more prior changes, creating more `git blame` targets per commit. If SZZ misattributes more for larger changesets, the measurement error could be differential — inflating the apparent defect rate for spec'd PRs. Two observations partially mitigate this concern: the rework measure (H2) does not depend on SZZ and shows the same directional pattern, and propensity score matching on size and complexity features eliminates the defect association entirely. Nonetheless, the reversed H1 association (specs predict *more* bugs) should be interpreted cautiously — confounding by indication and differential SZZ noise are difficult to disentangle.

Quality scoring is automated and unvalidated. Our LLM-based rubric (Claude Haiku) measures *formal* quality — whether the text contains outcome descriptions, error states, and acceptance criteria — not *functional* quality. A spec can score highly and still describe the wrong behavior. The rubric's seven dimensions align with what SDD tools prescribe, which means our scoring shares whatever blind spots those templates have. The scoring was applied only to PRs with substantial description text, creating a non-random subsample.

AI detection relies on voluntary tagging. We identify AI-assisted PRs via self-reported markers in the PR body. Developers who use AI without attribution are invisible. The false-negative rate is unknown. If many untagged PRs involved AI, the H5 comparison would be diluted toward the null.

The control group may be contaminated. Some "unspec'd" PRs may have had specifications shared verbally, in Slack, or in external tools. This would attenuate any treatment effect. However, if informal specification produces equivalent outcomes, the SDD value proposition still fails: vendors claim the *tooling and artifact* add value beyond what any structured thinking would provide.

Confounding by indication is the primary threat. Developers spec hard tasks. Hard tasks produce defects. The within-author design addresses this by comparing each developer to themselves, but within-author task selection remains possible: if an author consistently specs their hardest work and skips specs on easy tasks, the comparison is biased. The JIT-controlled models and propensity score matching (both in Section 5.6) address this further, and both confirm the null for defects.

Time-varying confounding. An author's skill, tooling, and project context may change over the observation period. The within-author estimator assumes these are stable.

7.3 External Validity

Open-source only. All 119 repositories are open-source. SDD tools target commercial teams with product managers, QA processes, and sprint planning. The specification practices and defect economics may differ.

Convenience sample. Repositories were not randomly selected. Results cannot be generalized beyond the sample.

Pre-agentic era. Most PRs predate widespread agentic AI coding tools. The dataset contains 5,989 AI-tagged PRs (after bot exclusion) — the closest available proxy for agentic workflows. Within this subsample, specs show no effect on defects ($p = 0.424$) or rework ($p = 0.399$). We cannot rule out that true agentic workflows would produce different results — but that is itself an untested claim.

7.4 Conclusion Validity

Multiple comparisons. We test five hypotheses without correction. At $\alpha = 0.05$, approximately 0.25 false positives are expected by chance.

Power for AI interaction. The scope-constraint interaction (H5) is identified from 366 authors with variation in both specification and AI use. The null result ($p = 0.997$) is not a power issue — the coefficient is effectively zero.

No pre-registration. The five hypotheses, operationalizations, quality thresholds, and robustness checks were not pre-registered. All analyses are post-hoc. We cannot rule out that alternative analyst choices (different quality thresholds, different rework definitions, different control sets) would produce different results. The ten robustness checks are intended to address this concern by showing stability across specifications, but a pre-registered confirmatory study would provide stronger evidence.

8 Conclusion

Spec-driven development tools make five specific, testable claims: specifications reduce defects (H1), prevent rework (H2), improve outcomes through higher-quality requirements (H3, H4), and

constrain AI-generated code scope (H5). We test the claims using the best available proxy — purpose-built SDD tool data is not publicly available — analyzing 88,052 pull requests across 119 open-source repositories, where specification artifacts are scored on the same quality dimensions that SDD tools prescribe, using within-author fixed-effects estimation.

None of the five hypotheses are supported at a level that would justify the vendor claims. The unadjusted association between specifications and defects is reversed (specifications accompany *more* defects); after within-author controls, the most parsimonious interpretation is confounding by indication. Specification quality has no meaningful effect on rework ($p = 0.827$). Specifications do not constrain AI-generated code scope ($p = 0.997$). Ten robustness checks confirm the null. Specification presence proxies for task complexity, not for quality improvement.

Three important caveats bound these findings. First, we test organic specification artifacts, not SDD tool-generated specifications — the construct gap is real (Section 7). Second, SZZ defect tracing has substantial measurement noise (46–71% misattribution), which attenuates true effects. Third, our open-source convenience sample may not generalize to commercial teams. Whether purpose-built SDD tooling would produce different results on commercial codebases remains an open question. The best available proxy evidence offers no support for the quality claims vendors are making.

Specifications may still have value — but not the value being claimed. A specification is an auditable record of what the code was *meant* to do, which is valuable for compliance, debugging, and onboarding regardless of whether it prevents defects. Specifications may improve developer efficiency by enabling task batching: a developer can queue structured work for an AI agent and shift attention elsewhere, reclaiming time without improving code quality. Specifications create accountability, making it harder for defects to pass unchallenged through review. These are real benefits. They are not "fewer defects" or "reduced rework." SDD vendors would do well to make claims they can substantiate.

The dataset and analysis code are available at: `github.com/brennhill/delivery-gap-research`.

9 References

Angrist, J. D., & Pischke, J.-S. (2009). *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.

Beck, K., Beedle, M., Bennekum, A. van, Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001,). *Manifesto for Agile Software Development*. https://agilemanifesto.org/

Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall.

Brooker, M. (2025, July 15). *Kiro and the Future of AI Spec-Driven Software Development*. https://kiro.dev/blog/kiro-and-the-future-of-software-development/

Böckeler, B. (2025, October 15). *Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl*. https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

Costa, D. A. da, McIntosh, S., Shang, W., Kuber, U., Coelho, R., & Hassan, A. E. (2017). A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes. *IEEE Transactions on Software Engineering*, *43*(7), 641–657.

Delimarsky, D. (2025, September 2). *Spec-Driven Development with AI: Get Started with a New Open Source Toolkit*. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/

Eberhardt, C. (2025, November 26). *Putting Spec Kit Through Its Paces: Radical Idea or Reinvented Waterfall?*. https://blog.scottlogic.com/2025/11/26/putting-spec-kit-through-its-paces-radical-idea-or-reinvented-waterfall.html

Femmer, H., Fernández, D. M., Wagner, S., & Eder, S. (2017). Rapid Quality Assurance with Requirements Smells. *Journal of Systems and Software*, *123*, 190–213.

GitHub Copilot Academy. (2025, September). *Spec-Driven Development*. https://copilot-academy.github.io/workshops/immersive-experience/spec_driven_development

Griffin, L., & Carroll, R. (2026, January 12). *Spec Driven Development: When Architecture Becomes Executable*. https://www.infoq.com/articles/spec-driven-development/

Guo, H., Ji, Y., & Zhou, X. (2025). Sedeve-Kit, a Specification-Driven Development Framework for Building Distributed Systems. *Arxiv Preprint*. https://arxiv.org/abs/2509.11566

Kamei, Y., Shihab, E., Adams, B., Hassan, A. E., Mockus, A., Sinha, A., & Ubayashi, N. (2013). A Large-Scale Empirical Study of Just-in-Time Quality Assurance. *IEEE Transactions on Software Engineering*, *39*(6), 757–773.

Marri, S. R. (2026). Constitutional Spec-Driven Development: Enforcing Security by Construction in AI-Assisted Code Generation. *Arxiv Preprint*. https://arxiv.org/abs/2602.02584

Mund, J., Femmer, H., Fernández, D. M., & Eckhardt, J. (2015). Does Quality of Requirements Specifications Matter? Combined Results of Two Empirical Studies. *Proceedings of the 9th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 1–10.

Naszcyniec, R. (2025, October 22). *How Spec-Driven Development Improves AI Coding Quality*. https://developers.redhat.com/articles/2025/10/22/how-spec-driven-development-improves-ai-coding-quality

Neyman, J., & Scott, E. L. (1948). Consistent Estimates Based on Partially Consistent Observations. *Econometrica*, *16*(1), 1–32.

Piskala, D. B. (2026). Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants. *Arxiv Preprint*. https://arxiv.org/abs/2602.00180

Roy, Y. (2026). The Kitchen Loop: User-Spec-Driven Development for a Self-Evolving Codebase. *Arxiv Preprint*. https://arxiv.org/abs/2603.25697

Salas, M., Hofman, A., & Stricker, B. H. C. (1999). Confounding by Indication: An Example of Variation in the Use of Epidemiologic Terminology. *American Journal of Epidemiology*, *149*(11), 981–983.

Shull, F., Basili, V., Boehm, B., Brown, A. W., Costa, P., Lindvall, M., Port, D., Rus, I., Tesoriero, R., & Zelkowitz, M. (2002). What We Have Learned About Fighting Defects. *Proceedings of the 8th International Symposium on Software Metrics*, 249–258.

Yanchuleff, R. (2025, July 15). *From Chat to Specs: A Deep Dive into AI-Assisted Development with Kiro*. https://kiro.dev/blog/from-chat-to-specs-deep-dive/

Zaninotto, F. (2025, November 12). *Spec-Driven Development: The Waterfall Strikes Back*. https://marmelab.com/blog/2025/11/12/spec-driven-development-waterfall-strikes-back.html

Zietsman, C. (2026). The Specification as Quality Gate: Three Hypotheses on AI-Assisted Code Review. *Arxiv Preprint*. https://arxiv.org/abs/2603.25773

Śliwerski, J., Zimmermann, T., & Zeller, A. (2005). When Do Changes Induce Fixes?. *Proceedings of the 2nd International Workshop on Mining Software Repositories (MSR)*, 1–5.